Python Programming

Chapter 2 notes

> Sanved

Loops

Loops are used to repeat a block of code multiple times.

for loop

syntax: ```python for i in iterable: block of code ```

In [ ]:
```
arr = [1, 2, 3, 4, 5]
for i in arr:
    print(i, end=' ')
```

```
1 2 3 4 5
```

enumerate() syntax:

```
for i, item in enumerate(iterable):
    block of code
```

It returns the index and the item of the iterable.

In [ ]:
```
arr = [10, 20, 30, 40, 50]
for i, v in enumerate(arr):
    print(f"index: {i}, value {v}")
```

```
index: 0, value 1
index: 1, value 2
index: 2, value 3
index: 3, value 4
index: 4, value 5
```

In [ ]:
```python
for i, v in [(0, 10), (1, 20), (2, 30), (3, 40), (4, 50)]:
    print(f"{i}, {v}")
```

```
0, 10
1, 20
2, 30
3, 40
4, 50
```

In [ ]:
```python
nested_arr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for i in nested_arr:
    for j in i:
        print(j, end=' ')
    print()
```

```
1 2 3
4 5 6
7 8 9
```

In [ ]:
```python
parties = ["bjp", "congress", "dmk"]
constituencies = [240, 100, 10]
for party, constituency in zip(parties, constituencies):
    print(f"{party} has {constituency} seats")
```

```
bjp has 240 seats
congress has 100 seats
dmk has 10 seats
```

In [ ]:
```python
list(zip(parties, constituencies))
```

Out[ ]:
```
[('bjp', 240), ('congress', 100), ('dmk', 10)]
```

In [ ]:
```python
a = [1, 2, 3, 4, 5]
b = [10, 20, 30, 40, 50, 60]
```

```
print(list(zip(a, b)))
print(list(zip(b, a)))
```

```
[(1, 10), (2, 20), (3, 30), (4, 40), (5, 50)]
[(10, 1), (20, 2), (30, 3), (40, 4), (50, 5)]
```

In [ ]:
```
c =  ['first', 'second', 'third', 'fourth', 'fifth']
print(list(zip(a, b, c)))
```

```
[(1, 10, 'first'), (2, 20, 'second'), (3, 30, 'third'), (4, 40, 'fourth'), (5, 50, 'fifth')]
```

In [ ]:
```
a = [1, 2, 3]
b = ['a', 'b', 'c']
for index, (x, y) in enumerate(zip(a, b)):
    print(f"i: {index}\t values: {x}, {y}")
```

```
i: 0     values: 1, a
i: 1     values: 2, b
i: 2     values: 3, c
```

For...else

```
for i in iterable:
    block of code
else:
    block of code
```

In [ ]:
```
for i in range(10):
    print(i, end=' ')
else:
    print("loop completed")
```

```
0 1 2 3 4 5 6 7 8 9 loop completed
```

while loop

syntax: ```python while condition: block of code ``` We use the while loop when we don't know the number of iterations.

In [ ]:
```
i = 0
arr = [1, 2, 3, 4, 5]
```

```
while i < len(arr):
    print(arr[i], end=' ')
    i += 1
```

```
1 2 3 4 5
```

In [ ]:
```
i = 10
j = 5
while i > 5:
    while j > 2:
        print(i, j)
        j -= 1
    print(i)
    i -= 1
```

```
10 5
10 4
10 3
10
9
8
7
6
```

Break, Continue, and Pass

Break

Break is used to exit a loop when a certain condition is met. Syntax:

```
for i in iterable:
    if condition:
        break
```

In [ ]:
```
team_scores = [12, 23, 54, 26, 76, 31]
i = 0; total_score = 0
while i < len(team_scores):
    total_score += team_scores[i]
    print(f"total score till iteration {i}: {total_score}")
    if total_score > 100:
```

```
        print(f"breaking the loop at iteration {i}")
        break
    i += 1
```

```
total score till iteration 0: 12
total score till iteration 1: 35
total score till iteration 2: 89
total score till iteration 3: 115
breaking the loop at iteration 3
```

In [ ]:
```
arr = [1, 2, 3, 4, 5]
sum = 0
for i in arr:
    print(f"{arr[i]}", end=' ')
    sum += arr[i]
    if sum > 5:
        break
```

```
2 3 4
```

Continue

Continue is used to skip the current iteration and move to the next iteration. Syntax:

```
for i in iterable:
    if condition:
        continue
```

In [ ]:
```
for i in range(10):
    if i % 2 == 0:
        continue
    print(i, end=' ')
```

```
1 3 5 7 9
```

Pass

Pass is used to do nothing. Syntax:

```
for i in iterable:
    pass
```

In [ ]:
```
grade_points = [
    [9.5, 8.3, 7.3, 5],
    [8.5, 7.3, 6.3, 4],
    [7.5, 6.3, 5.3, 3]
]
for grades in grade_points:
    for cgpa in grades:
        pass
        # pass is used when we don't have any code to write in the block
```

Else in loops

In [ ]:
```
arr = [1, 2, 3, 4, 5]
for i in arr:
    print(i, end=' ')
else:
    print("loop completed")
```

```
1 2 3 4 5 loop completed
```

In [ ]:
```
arr = [1, 2, 3, 4, 5]
for i in arr:
    print(i, end=' ')
    if i == 3:
        break
else:
    print("3 is not present in the list")
```

```
1 2 3
```

The else block is executed when the loop is not terminated by a break statement. Syntax:

```
for i in iterable:
    block of code
else:
    block of code
```

If the else is not executed, it means the loop was terminated by a break statement.

Strings

String operators

In [ ]:
```
# concatenate
"hello " + "there"
```

Out[ ]:
```
'hello there'
```

In [ ]:
```
# repeat
"sanved" * 3
```

Out[ ]:
```
'sanvedsanvedsanved'
```

In [ ]:
```
# compare
# ASCII values are compared for strings character by character
# if the first character of the first string is greater than the first character of the second string,
# then the first string is greater
print("sanved" == "sanved")
print("sanved" == "Sanved")
print("sanved" > "Sanved")
print("sAnved" < "sanVed")
print("cat" > "articuno")
# in cat and articuno, c is compared with a,
# since c is greater than a, cat is greater than articuno
print(ord("c"), ord("a"))
```

```
True
False
True
True
True
99 97
```

In [ ]:
```
# membership
print("cat" in "articuno")
```

```
print("cat" not in "articuno")
print("cat" in "catastrophe")
```

```
False
True
True
```

In [ ]:

```
# escape characters
print("hello\nworld"); print()  # newline
print("hello\tworld"); print()  # tab
print("hello\"world"); print()  # double quote
print("hello'world"); print()   # single quote
print("hello\bworld"); print()  # backspace
print("hello\rworld"); print()  # carriage return
print("hello\\world"); print()  # backslash
print("hello\vworld"); print()  # vertical tab
print("hello\aworld"); print()  # alert
print("hello\0world"); print()  # null
print("hello\x48world"); print()    # hex
print("\141")  # octal
```

```
hello
world

hello   world

hello"world

hello'world

hellworld

world

hello\world

hello♪world

hello☎world

helloworld

helloHworld

a
```

In [ ]:
```python
# slicing
name = "sanved"
print(name[0])
print(name[1:3])
print(name[1:])
print(name[:3])
print(name[1:5:2])
print(name[::2])
print(name[:100])
print(name[::-1])
```

```
s
an
anved
san
av
sne
sanved
devnas
```

In [ ]:
```python
# string formatting
# % operator
print("%d sheep, %s sheep" % (3, "wild"))
# format method
print("{} sheep, {} sheep".format(3, "wild"))
print("{0} sheep, {1} sheep".format(3, "wild"))
print("{1} sheep, {0} sheep".format(3, "wild"))
print("{count} sheep, {type} sheep".format(count=3, type="wild"))   # keyword arguments
# NOTE: keyword arguments should be passed after positional arguments
# f-string
print(f"{3} sheep, {'wild'} sheep")
count = 3; type = "wild"
print(f"{count} sheep, {type} sheep")
print(F"{count + 1} sheep, {type} sheep")
```

```
3 sheep, wild sheep
3 sheep, wild sheep
3 sheep, wild sheep
wild sheep, 3 sheep
3 sheep, wild sheep
3 sheep, wild sheep
3 sheep, wild sheep
4 sheep, wild sheep
```

In [ ]:
```python
# integer formatting
print("{:d} sheep".format(4))   # no type conversion here, so can't pass a float like 4.0
print("{:5d} sheep".format(4))  # right aligned
print("{:<5d} sheep".format(4)) # left aligned
print("{:^5d} sheep".format(4)) # center aligned
print("{:05d} sheep".format(4)) # zero padding
print("{:+d} sheep".format(4))  # sign
```

```
print(f"{10:d} sheep")
print(f"{10:5d} sheep")
```

```
4 sheep
    4 sheep
4     sheep
  4   sheep
00004 sheep
+4 sheep
10 sheep
   10 sheep
```

In [ ]:
```
# float formatting
print("{:f} sheep".format(4.0)) # default precision is 6    # 4.000000 sheep
print("{:.2f} sheep".format(4.0))   # 2 decimal places  # 4.00 sheep
print("{:10.2f} sheep".format(4.0)) # right aligned # ______4.00 sheep
print("{:<10.2f} sheep".format(4.0))    # left aligned # 4.00______ sheep
print("{:^10.2f} sheep".format(4.0))    # center aligned # __4.00____ sheep
print("{:010.2f} sheep".format(4.0))    # zero padding # 0000004.00 sheep
print("{:+.2f} sheep".format(4.0))  # sign # +4.00 sheep
print("{:.0f} sheep".format(4.0))   # no decimal places # 4 sheep

print(f"{4.0:f} sheep")
print(f"{4.0:.2f} sheep")
```

```
4.000000 sheep
4.00 sheep
      4.00 sheep
4.00       sheep
   4.00    sheep
0000004.00 sheep
+4.00 sheep
4 sheep
4.000000 sheep
4.00 sheep
```

In [ ]:
```
# string formatting
print("{:s} sheep".format("wild"))
print("{:10s} sheep".format("wild")) # right aligned
print("{:<10s} sheep".format("wild"))    # left aligned
print("{:^10s} sheep".format("wild"))    # center aligned
```

```
print("{:010s} sheep".format("wild"))    # zero padding
print("{:*^10s} sheep".format("wild"))   # character padding

print()

print("{:.3}".format("wild"))   # truncating
print("{:_^10.3}".format("wild")) # right aligned
print("{:_<10.3}".format("wild"))    # left aligned
```

```
wild sheep
wild       sheep
wild       sheep
   wild    sheep
wild000000 sheep
***wild*** sheep

wil
___wil____
wil_______
```

In [ ]:
```
# f-strings can have multiple lines
print(f"""
    a cat says "meow"
     a fox says "ring-ding-ding-ding-dingeringeding"
      a squid doesn't say anything
      """)
```

```
   a cat says "meow"
    a fox says "ring-ding-ding-ding-dingeringeding"
     a squid doesn't say anything
```

In [ ]:
```
print("binary: {:b}".format(10))
print("octal: {:o}".format(10))
print("hex: {:x}".format(10))
print("hex: {:X}".format(10))
print("scientific: {:e}".format(10))
print("scientific: {:E}".format(10))
print("percentage: {:.2%}".format(0.1))
print("percentage: {:10.2%}".format(0.1))
```

```
binary: 1010
octal: 12
hex: a
hex: A
scientific: 1.000000e+01
scientific: 1.000000E+01
percentage: 10.00%
percentage:     10.00%
```

In [ ]:
```python
print(1,000 + 2,000)   # this is just 1, 0+2, 0
print(1_000 + 2_000)   # _ is ignored by the interpreter
print(1,00 + 2_00)
```

```
1 2 0
3000
1 200
```